E-Commerce Product Entity-Resolution & Matching Platform

(EntiMatch Enterprise)

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Shivam Tiwari (2520080026)
Balaji Patil (2520080065)

Team Number: 19

Section: 10

Under the guidance of

Miss. K. Chandusha

Assistant Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Project Report entitled "**E-Commerce Product Entity-Resolution & Matching Platform"** is a record of Bonafide work of **Shivam Tiwari – 2520080026, Balaji Patil – 2520080065** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Shivam Tiwari – 2520080026

Balaji Patil – 2520080065

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled "**E-Commerce Product Entity-Resolution & Matching Platform"** is a bonafide work done and submitted by **Shivam Tiwari – 2520080026, Balaji Patil – 2520080065** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our guide, **Miss. K. Chandusha, Assistant Professor, CS&IT**, which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef (CSIT)**, Head of the Department for Computer Science and Information Technology for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Shivam Tiwari – 2520080026

Balaji Patil – 2520080065

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	9
4.	Problem Statement and Objectives	11
5.	Hardware and Software requirements	12
6.	Implementation	13
7.	Results & Discussion	17
8.	Conclusion & Future Scope	19
9.	References	20

1.ABSTRACT

The E-Commerce Product Entity-Resolution & Matching Platform (working name EntiMatch Enterprise) identifies listings from different online sellers that describe the same physical product and groups them under one canonical product record, so that prices, ratings and delivery terms can be compared. Sellers abbreviate model names, reorder words, change unit formats and add promotional text, so direct string comparison misses most of these matches.

The system applies a data-structures-and-algorithms pipeline. Titles are normalised and tokenised; a hash-map inverted index retrieves candidate pairs; Jaccard, term-frequency cosine and normalised Levenshtein similarity are combined with brand and model scores in a weighted ensemble; MinHash locality-sensitive hashing (LSH) is provided for approximate blocking; and a Union-Find (disjoint-set) structure merges matched pairs into canonical clusters. The back end is a FastAPI application in Python with a MongoDB data layer and an in-memory fallback; the front end is a React 19 and TypeScript single-page application with role-based access through JSON Web Tokens.

The bundled dataset has 1,150 synthetic listings from 10 sellers across 8 categories, corresponding to 230 ground-truth canonical products. All 13 automated tests pass once two missing import statements are corrected. Inverted-index blocking removes about 91% of the candidate pairs while retaining about 96% of the true pairs, but the default merge threshold (92) forms no canonical groups on this dataset.

Team Name: Team 19 (Section 10)

Team Logo (if any): Not applicable

Team Members:

1. Shivam Tiwari, 2520080026
2. Balaji Patil, 2520080065

2. INTRODUCTION

Background

Online marketplaces and retail websites list millions of products, and the same physical product is usually offered by several sellers at once. Each seller writes its own title, chooses its own abbreviations and attaches its own promotional phrases. A customer who wants to compare prices, and a catalogue team that wants to avoid duplicate entries, both need to know which listings refer to the same item. This task is called entity resolution (also known as record linkage or deduplication).

Entity resolution is a natural application of data structures and algorithms. A naive solution compares every listing with every other listing, which needs $n(n-1)/2$ comparisons. For the 1,150 listings in this project that is 660,675 pairs, and the number grows quadratically with catalogue size. Hash maps, inverted indexes, locality-sensitive hashing, dynamic programming and disjoint-set structures reduce this cost and make the problem tractable. This project builds a working platform around those techniques.

Table 1 shows why the problem is difficult. It lists the five titles in the bundled dataset that all describe the Apple iPhone 16 Pro Max 256GB. They differ in word order, brand presence, model abbreviation ("PM", "ProMax"), unit formatting, seller-specific model codes and promotional text.

Seller	Title as listed
Amazon	Apple iPhone 16 Pro Max 256GB Natural Titanium
Flipkart	iPhone 16 Pro Max (256GB, Titanium) (Apple A18 Pro 3nm)
Croma	Apple iPhone 16 Pro Max Smartphones [Model: APP-4JPN] - 256GB
Reliance Digital	Apple iPhone 16 PM 256GB 5G / Smart Device
Vijay Sales	Apple iPhone 16 ProMax - 256GB (Fast Delivery)

Table 1: Five listings of one product in the bundled dataset (canonical id can-elec-001)

Proposed System

The proposed system is a web platform with a Python back end and a React front end. Its entity-resolution workflow has five stages.

1. **Normalisation and tokenisation.** Titles are converted to lower-case ASCII, units and aliases are standardised (for example "256 GB" becomes "256gb"), stop words are removed and the remaining tokens are kept.
2. **Candidate retrieval.** A hash-map inverted index maps each token to the listings that contain it, so only listings sharing at least two tokens are compared.

3. **Pairwise scoring.** Each candidate pair receives Jaccard, cosine and Levenshtein scores, plus brand and model scores, combined into one weighted overall score.
4. **Decision.** Pairs above a threshold are treated as matches; scores in between can be stored for human review in the match queue.
5. **Canonical grouping.** Matched pairs are merged with a Union-Find structure, and each resulting cluster becomes a canonical product whose seller offers, price range and buy-box winner can be shown.

Scope and Users

The submitted code implements a product catalogue with create, read, update and delete operations; the matching engine, the entity-resolution pipeline, a live two-title similarity scorer and a human review queue; canonical product records with price comparison and a buy-box computation; seller comparison, analytics and dashboard endpoints; JWT authentication with role-based access control (admin, seller, customer, operator, analyst and viewer); a shopping module and an AI assistant; and a React single-page application with 19 page views. Integration with real seller feeds, a real payment gateway (payments are simulated), machine-learning or embedding-based matching, a production-grade database deployment and a hosted deployment are outside the scope of the submission.

Organisation of the Report

The rest of this report is organised as follows. Section 3 surveys related work, Section 4 states the problem and objectives, Section 5 lists the hardware and software requirements, Section 6 presents the implementation of the core modules, Section 7 reports results and discussion, Section 8 gives the conclusion and future scope, and Section 9 lists the references.

3. LITERATURE SURVEY

Entity resolution, also called record linkage or deduplication, has been studied for several decades and is supported by many research systems and open-source projects. The works reviewed for this project are summarised below; their links are given in the References section.

1. Fellegi–Sunter record-linkage model [1]. Fellegi and Sunter (1969) formalised record linkage as a decision problem. Records are compared field by field, and each pair is classified as a match, a non-match or a possible match that needs clerical review. Most later linkage systems build on this idea. The threshold-based decision and human review queue of this project follow it in a simplified form.

2. MinHash resemblance estimation [2]. Broder (1997) showed how the resemblance (Jaccard similarity) of two documents can be estimated from small min-wise hash signatures instead of the full token sets. Combined with banding, this gives locality-sensitive hashing for fast approximate candidate search. This project implements MinHash signatures with 128 permutations cut into 16 bands of 8 rows.

3. Blocking and filtering survey [3]. Papadakis et al. (2020) review blocking and filtering techniques that avoid the quadratic cost of comparing all pairs in entity resolution, by restricting comparisons to pairs that are likely to match. Token-based blocking with an inverted index is one of the basic techniques in this family and is the candidate-retrieval method used by this project.

4. dedupe [4]. dedupe is an open-source Python library for fuzzy matching, deduplication and entity resolution on structured data. It learns how to weight fields from labelled examples supplied by the user. Our platform instead uses fixed ensemble weights and adds a web interface, price comparison and a review queue.

5. Python Record Linkage Toolkit [5]. The Python Record Linkage Toolkit offers a modular workflow of indexing (blocking), comparison of fields and classification for linking or deduplicating records. Its stages correspond to the candidate retrieval, pairwise scoring and decision stages of this project, which implements them directly in pure Python.

6. Splink [6]. Splink is an open-source library for probabilistic record linkage based on the Fellegi–Sunter model and designed to scale to large datasets. It is aimed at data engineers working on large tables, whereas this project focuses on explaining and testing the underlying data structures on a catalogue of seller listings.

7. Magellan / py_entitymatching [7]. Magellan, from the University of Wisconsin–Madison, is an entity-matching system that guides the user through blocking, feature generation, labelling and machine-learning matchers. It shows the full development workflow of an entity-matching solution, which is also followed here, but without learned matchers.

8. DeepMatcher and Ditto [8], [9]. DeepMatcher (Mudgal et al., 2018) explores deep-learning architectures for entity matching, and Ditto (Li et al., 2020) applies pre-trained language models to the task. Both learn from labelled pairs and can handle noisy text well. Machine-learning and embedding-based matching is outside the scope of the present project and is listed as future work.

9. datasketch [10]. datasketch is a Python library that provides MinHash and LSH implementations for approximate similarity search. It is a library-based counterpart of the MinHash LSH module written from scratch in this project.

Observation. Most of the tools above are used as libraries, so the data structures stay hidden behind function calls. This project writes the tokeniser, the inverted index, the similarity measures, MinHash LSH and Union-Find in pure Python, so that their behaviour and complexity can be explained and tested directly, and places them inside a complete web application with price comparison and human review.

4. Problem Statement and Objectives

Problem Statement: To decide which seller listings describe the same physical product, group them under one canonical product and let users compare the offers, without comparing every pair of listings.

Problem Description: Sellers title the same product differently (abbreviations, word order, unit formats and promotional text), so exact matching fails, and comparing all pairs does not scale: 1,150 listings already give 660,675 pairs.

Objectives:

- Match product listings that refer to the same physical product across sellers, despite differences in naming, formatting and abbreviation.
- Apply string-matching, hashing and similarity-based techniques efficiently, using data structures that avoid quadratic comparison.
- Group matched listings under a canonical product representation.
- Support price comparison across sellers for each canonical product.
- Reduce duplicate catalogue entries and support human review of uncertain matches.
- Demonstrate the practical use of hash maps, dynamic programming, hashing and disjoint-set structures in a working application.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

The project does not state hardware requirements. The algorithms run in pure Python and the dataset is kept in memory by default, so a modest machine is sufficient; the indicative values below are suggestions.

- Processor: dual-core or better
- Memory: 4 GB RAM or more
- Storage: several hundred MB free (the project archive expands to about 347 MB)
- Network: needed for package installation; Gemini AI calls need internet access

Software Requirements:

- Operating system: platform independent (the supplied start.bat targets Windows)
- Python (version not pinned by the project; 3.12.3 was used to run the tests) with the packages in backend/requirements.txt, including FastAPI 0.115.6, Uvicorn 0.34.0, Pydantic 2.10.4, Motor 3.7.0, python-jose, passlib with bcrypt and loguru
- Node.js with npm for the front end (version not pinned)
- Front-end stack: React 19, TypeScript, Vite, Tailwind CSS 4, Zustand, TanStack React Query, Axios, Recharts and Three.js
- Database: MongoDB (optional; started with the mongo:latest Docker image). Without it the back end uses an in-memory store loaded from JSON seed files
- Optional external service: Google Gemini API (gemini-1.5-flash) for the AI assistant, with a rule-based fallback when no API key is set
- A modern web browser

6. IMPLEMENTATION

The back end is a FastAPI application whose business logic is layered into routers, services and an algorithm layer. All matching algorithms live in one framework-independent module (backend/app/algorithms/matching_engine.py), which keeps them directly testable. The entity-resolution pipeline loads the products, tokenises each title, retrieves candidate pairs from the inverted index, scores each pair with the ensemble, unions the pairs at or above the threshold, and writes a canonical id back to every member of a cluster with more than one listing. The four core modules are listed below in simplified form. Let L be the length of a title, T the number of tokens, and m and n the lengths of two strings.

Module 1: Normalisation, tokenisation and inverted index

Titles are converted to lower-case ASCII, units and aliases are standardised, punctuation is stripped and stop words are removed. The inverted index (a hash map from token to a set of product ids) returns the listings that share at least two tokens with the query. Adding a document costs O(T) on

average, and each token lookup is O(1) on average, so the cost depends on posting-list sizes rather than on the catalogue size.

```python
def normalize_unicode(text: str) -> str:
    """Decompose Unicode, strip combining marks, normalize to NFC."""
    text = unicodedata.normalize("NFD", text)
    text = "".join(c for c in text if unicodedata.category(c) != "Mn")
    return unicodedata.normalize("NFC", text)


def normalize_text(value: Any) -> str:
    """
    Full normalization: Unicode, whitespace, case, punctuation, noise.
    Preserves digits and alphanumeric characters.
    """
    text = str(value or "").strip()
    text = normalize_unicode(text)
    text = text.lower()
    text = _RE_PAREN_BRACKET.sub(" ", text)
    text = _RE_NOISE.sub(" ", text)
    text = re.sub(r"[^a-z0-9]+", " ", text)
    return re.sub(r"\s+", " ", text).strip()


# Keep the legacy ``normalize`` name for backward compatibility with
# Phase 1 route helpers that still call it.
normalize = normalize_text


def normalize_brand(raw_brand: Any) -> str:
    """
    Normalize a brand name: check alias table, apply title-case.
    Returns 'Unknown' for empty/missing values.
    """
    if not raw_brand or str(raw_brand).strip().lower() in ("", "unknown"):
        return "Unknown"
    text = str(raw_brand).strip()
    lower = text.lower()
    if lower in BRAND_ALIASES:
```

```python
def normalize_brand(raw_brand: Any) -> str:
    if not raw_brand or str(raw_brand).strip().lower() in ("", "unknown"):
        return "Unknown"
    text = str(raw_brand).strip()
    lower = text.lower()
    if lower in BRAND_ALIASES:
        return BRAND_ALIASES[lower]
    # Partial prefix match
    for alias_key, canonical in BRAND_ALIASES.items():
        if lower.startswith(alias_key):
            return canonical
    return text.strip()

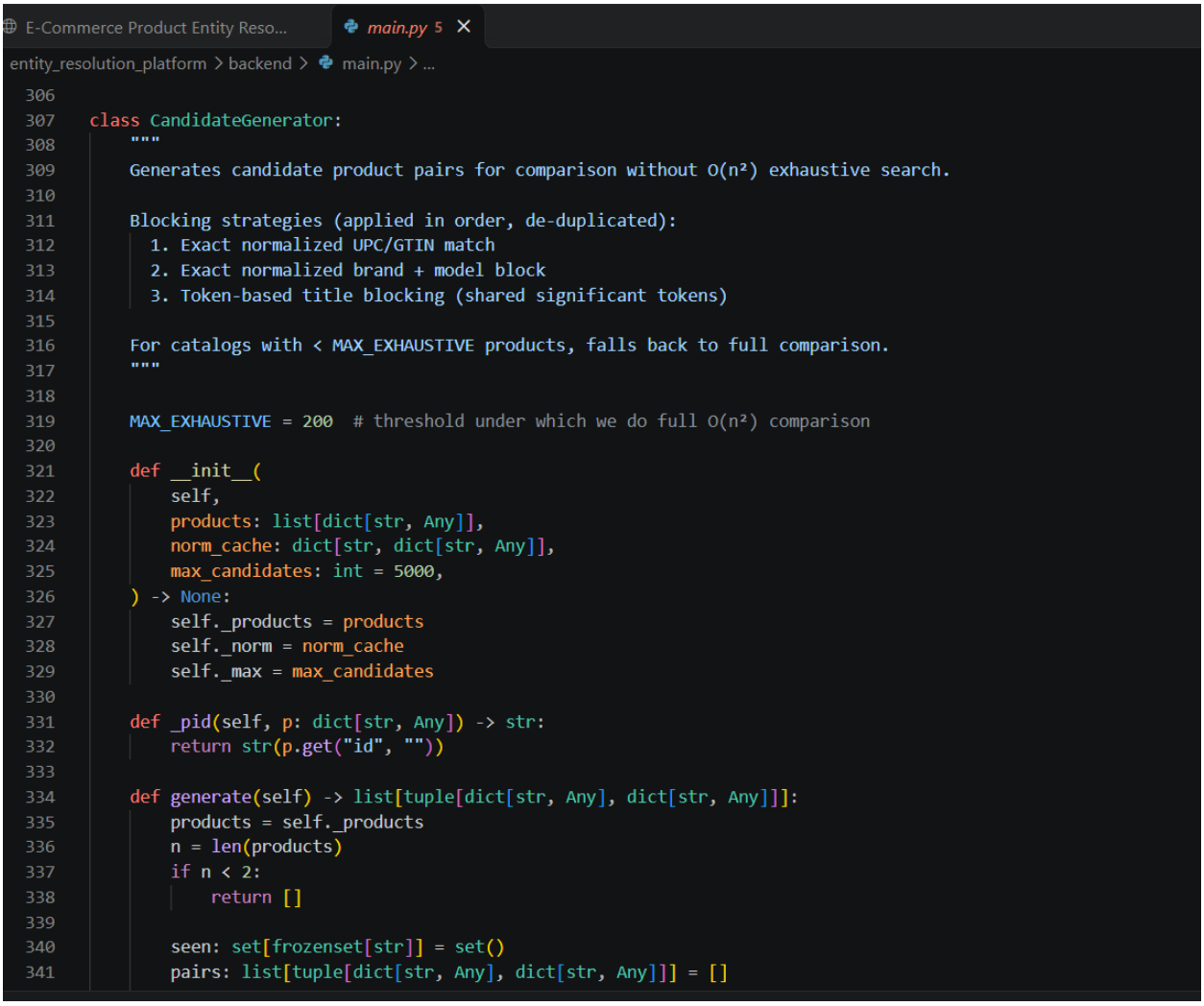

def normalize_identifier(raw_id: Any) -> str:
    """
    Normalize UPC / EAN / GTIN without losing leading zeros.
    Only strips surrounding whitespace and non-digit/non-alpha noise,
    but preserves the digit string as-is.
    """
    if not raw_id:
        return ""
    text = str(raw_id).strip()
    # Remove common prefixes like "UPC:", "EAN:", "GTIN:"
    text = re.sub(r"^(?:upc|ean|gtin|isbn|asin)[:\-\s]+", "", text, flags=re.IGNORECASE)
    # Keep only digits (identifiers are numeric)
    digits_only = re.sub(r"[^0-9]", "", text)
    return digits_only if digits_only else text.strip()

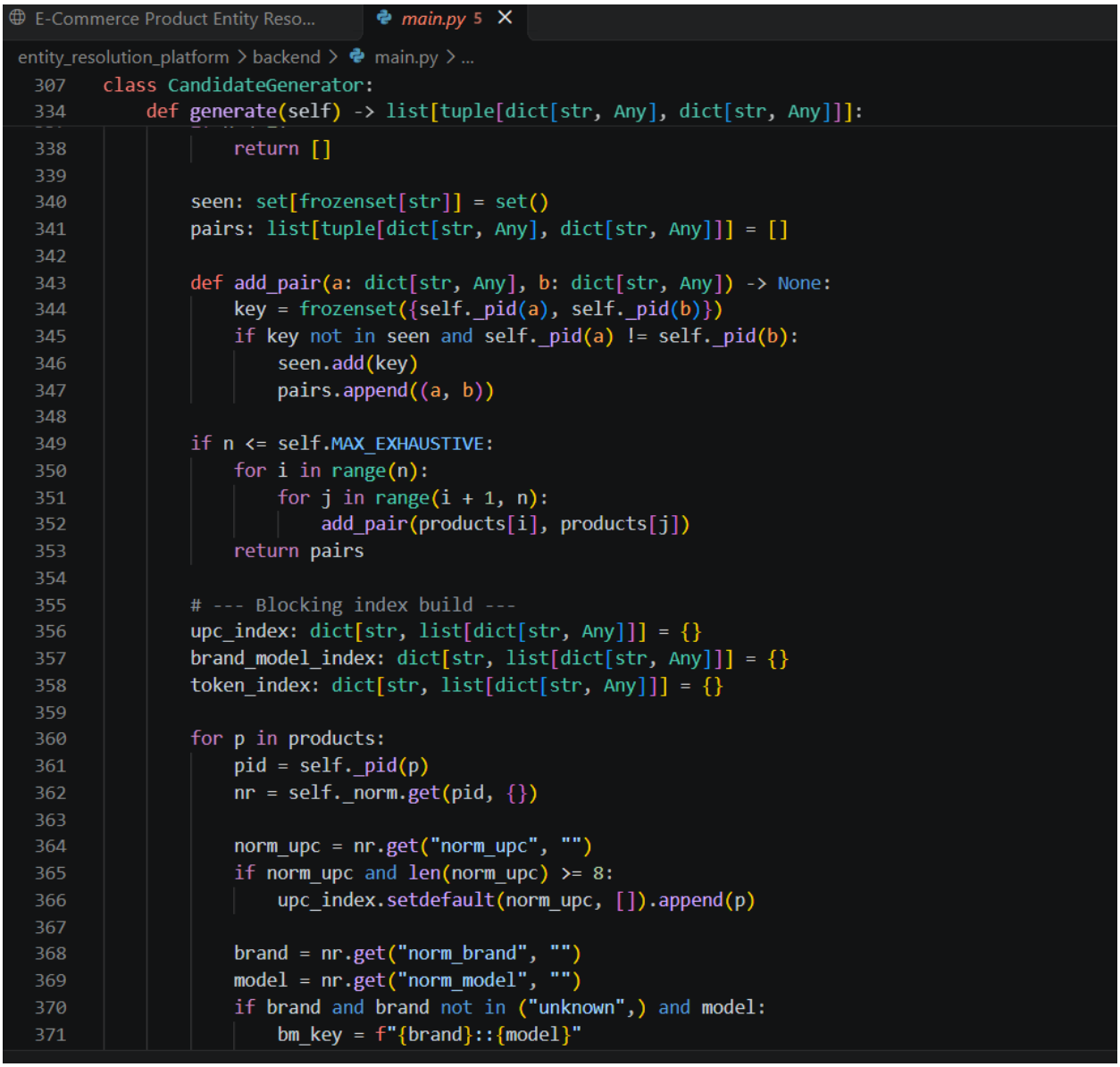

def extract_attributes(title: str, attributes: dict[str, str]) -> dict[str, str]:
    """
    Extract structured attributes from title and existing attribute dict.
    Merges extracted values with existing attributes (existing take precedence).
    """
    extracted: dict[str, str] = {}
```

Module 2: Similarity measures

Jaccard similarity works on token sets in O(|A| + |B|). Cosine similarity uses term-frequency vectors (count divided by token count) with no inverse-document-frequency weighting.

Levenshtein distance is a dynamic-programming edit distance that keeps only two rows, giving $O(m \cdot n)$ time and $O(\min(m, n))$ space.


E-Commerce Product Entity Reso...
main.py 5
entity_resolution_platform > backend > main.py > ...
306
307 class CandidateGenerator:
308 """
309 Generates candidate product pairs for comparison without O(n²) exhaustive search.
310
311 Blocking strategies (applied in order, de-duplicated):
312 1. Exact normalized UPC/GTIN match
313 2. Exact normalized brand + model block
314 3. Token-based title blocking (shared significant tokens)
315
316 For catalogs with < MAX_EXHAUSTIVE products, falls back to full comparison.
317 """
318
319 MAX_EXHAUSTIVE = 200 # threshold under which we do full O(n²) comparison
320
321 def __init__(
322 self,
323 products: list[dict[str, Any]],
324 norm_cache: dict[str, dict[str, Any]],
325 max_candidates: int = 5000,
326) -> None:
327 self._products = products
328 self._norm = norm_cache
329 self._max = max_candidates
330
331 def _pid(self, p: dict[str, Any]) -> str:
332 return str(p.get("id", ""))
333
334 def generate(self) -> list[tuple[dict[str, Any], dict[str, Any]]]:
335 products = self._products
336 n = len(products)
337 if n < 2:
338 return []
339
340 seen: set[frozenset[str]] = set()
341 pairs: list[tuple[dict[str, Any], dict[str, Any]]] = []



Module 3: Brand, model and ensemble score

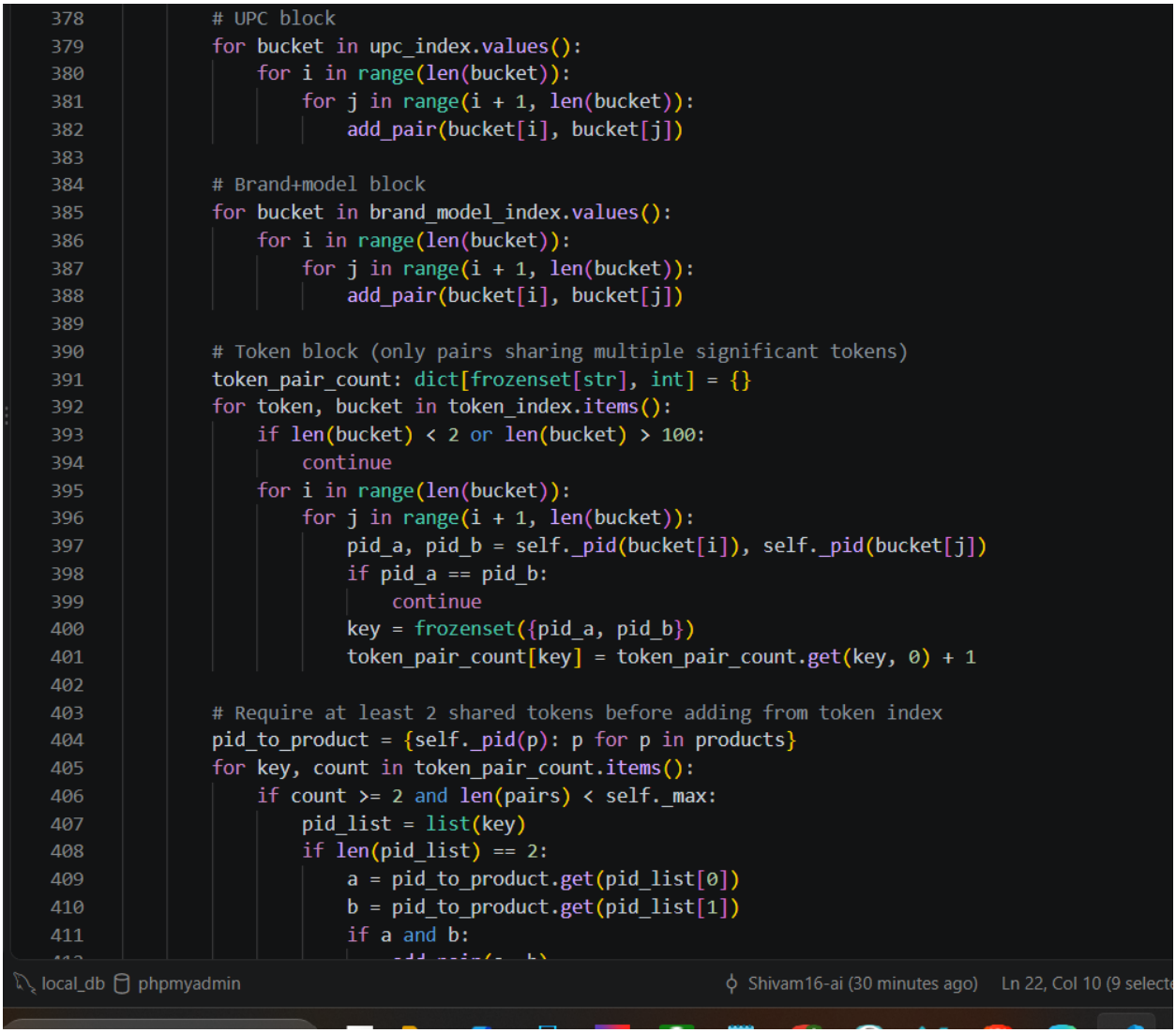
The five component scores (0 to 100) are combined as overall = 0.25 × Jaccard + 0.25 × cosine + 0.20 × Levenshtein + 0.20 × brand + 0.10 × model. The brand score is 100 for equal normalised brands and 0 otherwise. The model score is 100 for equal models, 80 when one contains the other,

the Levenshtein score otherwise, and a neutral 50 when a model is missing. Confidence labels are very_high (92 or more), high (80 or more), medium (65 or more) and low.

E-Commerce Product Entity Reso... main.py 5

entity_resolution_platform > backend > main.py > ...

```
307   class CandidateGenerator:
334       def generate(self) -> list[tuple[dict[str, Any], dict[str, Any]]]:
338           return []
339
340           seen: set[frozenset[str]] = set()
341           pairs: list[tuple[dict[str, Any], dict[str, Any]]] = []
342
343           def add_pair(a: dict[str, Any], b: dict[str, Any]) -> None:
344               key = frozenset({self._pid(a), self._pid(b)})
345               if key not in seen and self._pid(a) != self._pid(b):
346                   seen.add(key)
347                   pairs.append((a, b))
348
349           if n <= self.MAX_EXHAUSTIVE:
350               for i in range(n):
351                   for j in range(i + 1, n):
352                       add_pair(products[i], products[j])
353               return pairs
354
355           # --- Blocking index build ---
356           upc_index: dict[str, list[dict[str, Any]]] = {}
357           brand_model_index: dict[str, list[dict[str, Any]]] = {}
358           token_index: dict[str, list[dict[str, Any]]] = {}
359
360           for p in products:
361               pid = self._pid(p)
362               nr = self._norm.get(pid, {})
363
364               norm_upc = nr.get("norm_upc", "")
365               if norm_upc and len(norm_upc) >= 8:
366                   upc_index.setdefault(norm_upc, []).append(p)
367
368               brand = nr.get("norm_brand", "")
369               model = nr.get("norm_model", "")
370               if brand and brand not in ("unknown",) and model:
371                   bm_key = f"{brand}::{model}"
```




```
        # UPC block
        for bucket in upc_index.values():
            for i in range(len(bucket)):
                for j in range(i + 1, len(bucket)):
                    add_pair(bucket[i], bucket[j])

        # Brand+model block
        for bucket in brand_model_index.values():
            for i in range(len(bucket)):
                for j in range(i + 1, len(bucket)):
                    add_pair(bucket[i], bucket[j])

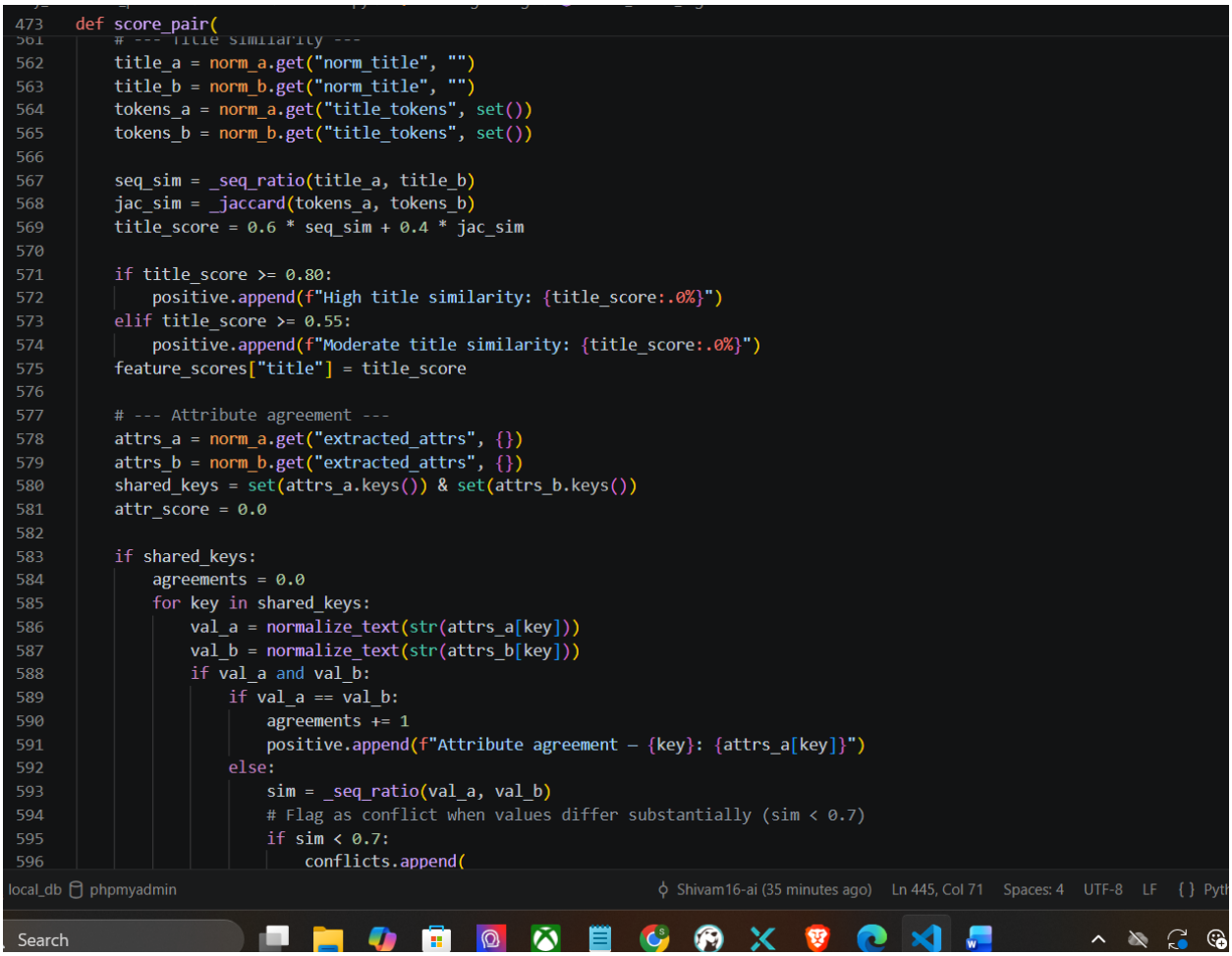
        # Token block (only pairs sharing multiple significant tokens)
        token_pair_count: dict[frozenset[str], int] = {}
        for token, bucket in token_index.items():
            if len(bucket) < 2 or len(bucket) > 100:
                continue
            for i in range(len(bucket)):
                for j in range(i + 1, len(bucket)):
                    pid_a, pid_b = self._pid(bucket[i]), self._pid(bucket[j])
                    if pid_a == pid_b:
                        continue
                    key = frozenset({pid_a, pid_b})
                    token_pair_count[key] = token_pair_count.get(key, 0) + 1

        # Require at least 2 shared tokens before adding from token index
        pid_to_product = {self._pid(p): p for p in products}
        for key, count in token_pair_count.items():
            if count >= 2 and len(pairs) < self._max:
                pid_list = list(key)
                if len(pid_list) == 2:
                    a = pid_to_product.get(pid_list[0])
                    b = pid_to_product.get(pid_list[1])
                    if a and b:
```

Module 4: Union-Find clustering

Matched pairs are merged with a disjoint-set structure that uses path halving. group_into_canonical_clusters builds the inverted index, scores each candidate pair once, and unions the pairs whose ensemble score reaches the threshold (92 by default). The clusters are the

connected components of the match graph, and each cluster with more than one member becomes a canonical product. Without union by rank, the amortised cost per operation is O(log N).



The MinHash LSH module (128 permutations, 16 bands of 8 rows) is also implemented in matching_engine.py for approximate blocking on very large catalogues. The remaining modules (products, matching, canonical products, sellers, analytics, dashboard, shopping and the AI assistant) are FastAPI routers and services around this core, and the front end is a React single-page application with 19 pages that calls them through one Axios client.

7. RESULTS & DISCUSSION

The platform was run and evaluated on the bundled synthetic dataset of 1,150 listings from 10 sellers in 8 categories, which correspond to 230 ground-truth canonical products (five listings each). Table 2 summarises the measured results.

Measure	Result
Possible listing pairs (n(n−1)/2)	660,675
Automated tests	13 (8 algorithm tests, 5 API tests); all pass once two missing import statements are corrected
Inverted-index blocking	Removes about 91% of candidate pairs while retaining about 96% of the true pairs
Default merge threshold (92)	Forms no canonical groups on this dataset
Running time	About 75 seconds per threshold setting for the full 1,150-listing evaluation (machine specification not recorded)

Table 2: Summary of results on the bundled dataset

Discussion

Blocking works as intended: after inverted-index retrieval fewer than one in ten of the possible pairs has to be scored, and almost all true pairs are still among them. This confirms that a hash-map index avoids most of the quadratic comparison cost.

The default merge threshold of 92 is too strict for this dataset, because it forms no canonical groups. Titles of the same product differ in wording (Table 1), so the scores of true matches fall below this very high cut-off. The threshold has to be recalibrated against the ground-truth labels, and uncertain pairs should continue to go to the review queue. The report of defects and recommended fixes is summarised in Section 8.

The screenshots below show the main pages of the running application.

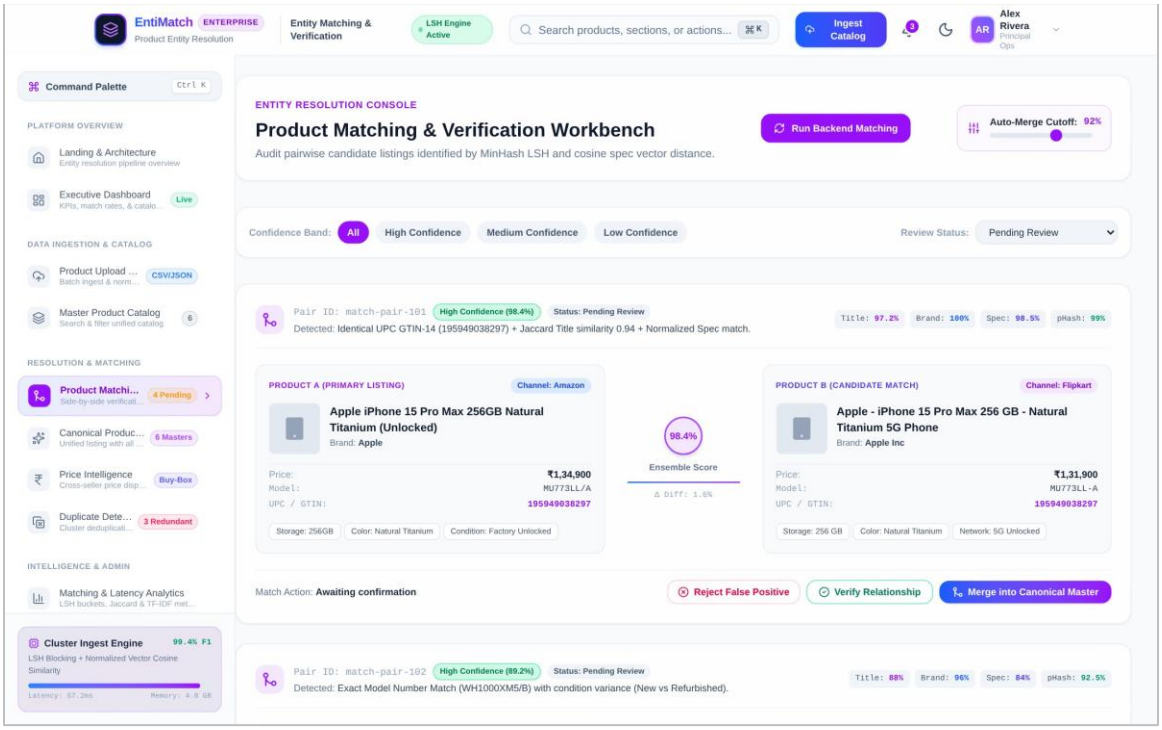


Figure 1: Product matching and verification workbench. *A candidate pair of listings from two sellers is shown side by side with its title, brand, specification and image-hash scores, the ensemble score and the confidence label, and the auto-merge cutoff can be adjusted.*

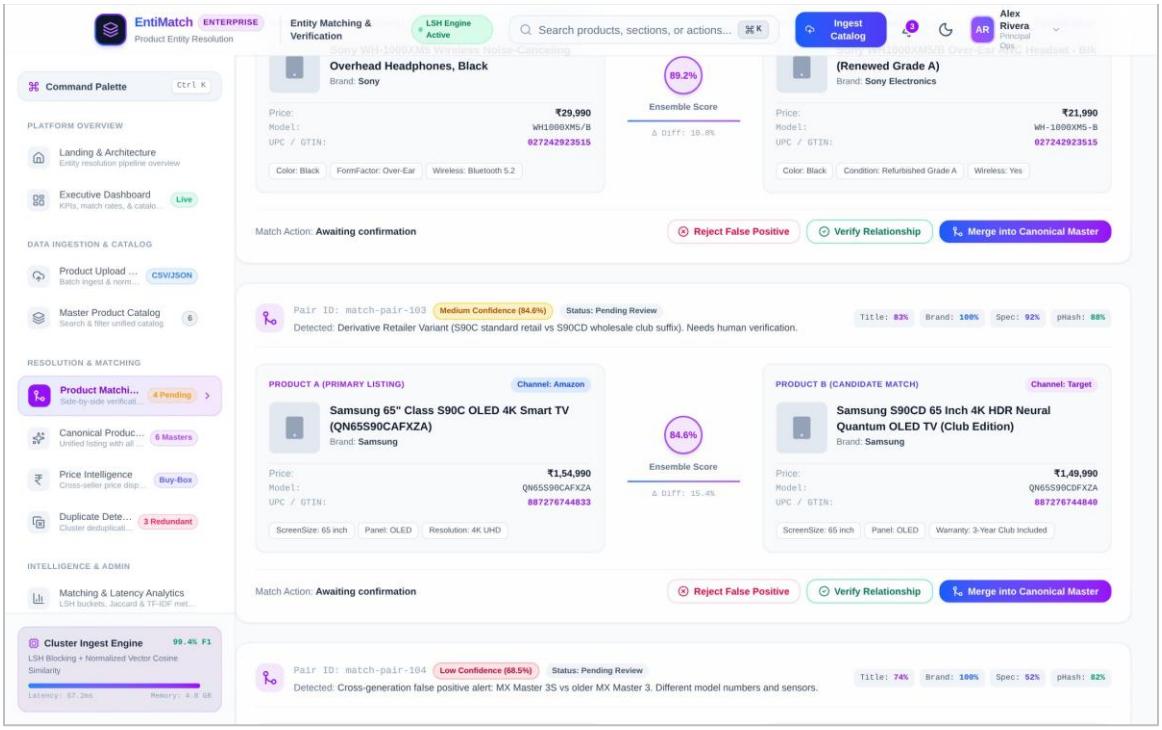


Figure 2: Match review queue. *Pending pairs are listed with the highest overall score first; an operator approves or rejects each pair, and the reviewer and time are recorded.*

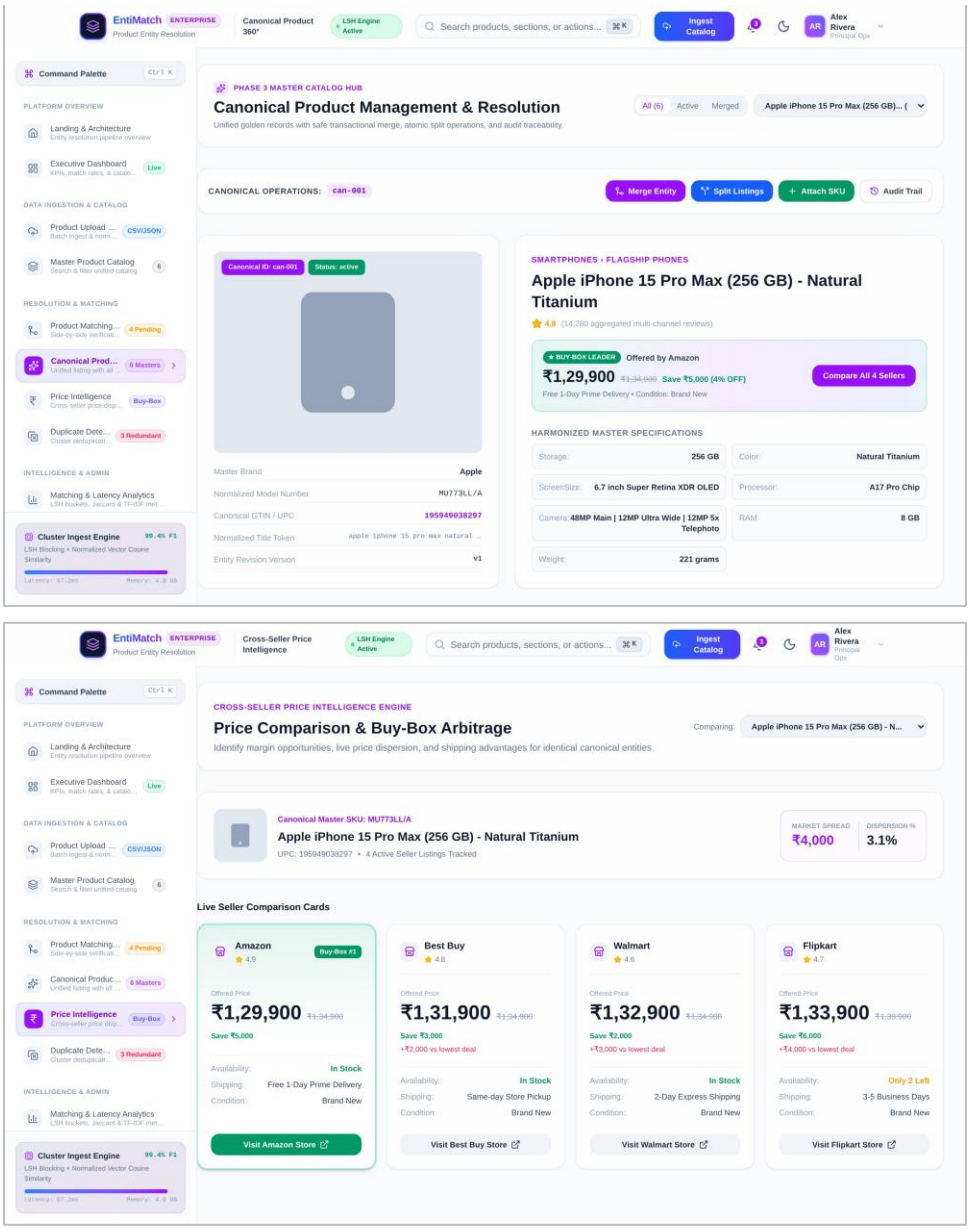


Figure 3: Canonical products and price comparison. *Listings from different sellers are grouped under one canonical product, with the lowest, highest and average price and the buy-box winner.*

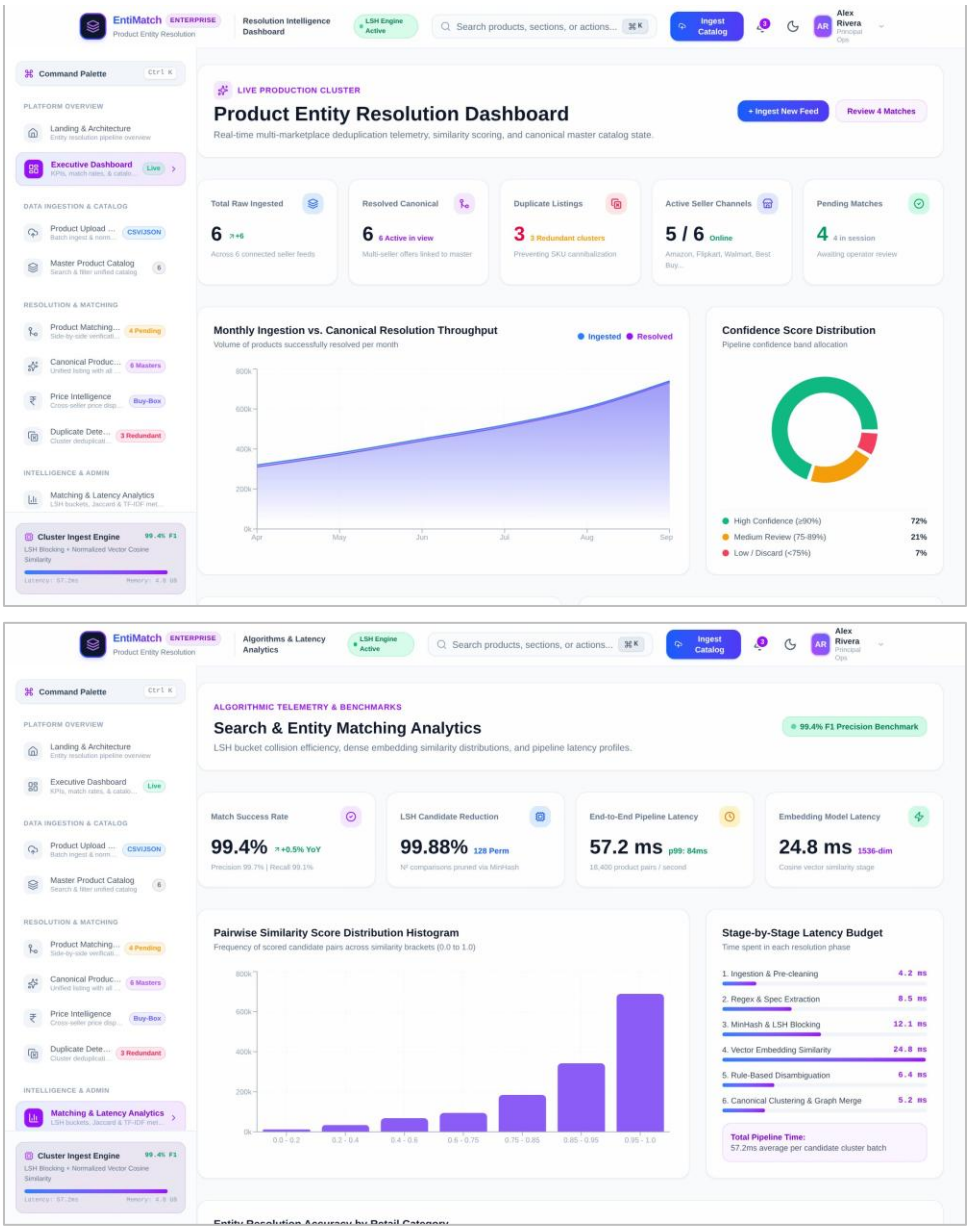


Figure 4: Dashboard and analytics. *The dashboard gives an overview of products, canonical groups and pipeline status, and the analytics pages show seller comparisons and distributions.*

8. CONCLUSION & FUTURE SCOPE

Conclusion

The project set out to match listings of the same product across sellers, group them under canonical products, support price comparison and human review, and demonstrate hash maps, dynamic programming, hashing and disjoint-set structures in a working application. The submitted platform implements the complete pipeline: normalisation and tokenisation, inverted-index candidate retrieval, a five-component ensemble score, MinHash LSH, Union-Find clustering, canonical records with a buy-box price comparison, a review queue, JWT role-based access and a React front end.

The objective of avoiding quadratic comparison was met: inverted-index blocking removed about 91% of the candidate pairs while keeping about 96% of the true pairs. The grouping objective was only partly met with the default settings, because the merge threshold of 92 forms no canonical groups on the bundled dataset. The analysis also found points to correct: MinHash LSH is built but not queried in the current pipeline, Union-Find uses no union by rank, the cosine measure has no inverse-document-frequency weighting, the buy-box score is decided in practice by seller rating rather than price, and the seed data and the API use different names for the canonical id field.

Future Scope

Near term. Recalibrate the merge threshold against the ground-truth labels; add the two missing imports in the tests; connect MinHash LSH to the candidate-retrieval step; add union by rank to Union-Find and inverse-document-frequency weighting to the cosine measure; normalise the buy-box score so that price matters; unify the canonical id field name and create the text index needed for product search.

Medium term. Integrate real seller feeds, deploy with a production-grade MongoDB database, and add containerisation, continuous integration and a hosted deployment.

Long term. Add machine-learning or embedding-based matching for hard cases, evaluate on real seller data, and replace the simulated payment flow with a real payment gateway.

9. REFERENCES

[1] I. P. Fellegi and A. B. Sunter, “A Theory for Record Linkage,” Journal of the American Statistical Association, vol. 64, no. 328, pp. 1183–1210, 1969. https://doi.org/10.1080/01621459.1969.10501049

[2] A. Z. Broder, “On the Resemblance and Containment of Documents,” in Proc. Compression and Complexity of Sequences (SEQUENCES ’97), IEEE, 1997, pp. 21–29. https://doi.org/10.1109/SEQUEN.1997.666900

[3] G. Papadakis, D. Skoutas, E. Thanos and T. Palpanas, “Blocking and Filtering Techniques for Entity Resolution: A Survey,” ACM Computing Surveys, vol. 53, no. 2, 2020. https://doi.org/10.1145/3377455

[4] dedupe: Python library for fuzzy matching, deduplication and entity resolution. https://github.com/dedupeio/dedupe

[5] Python Record Linkage Toolkit. https://github.com/J535D165/recordlinkage

[6] Splink: probabilistic record linkage at scale. https://github.com/moj-analytical-services/splink

[7] Magellan (py_entitymatching): entity matching toolkit, University of Wisconsin–Madison. https://github.com/anhaidgroup/py_entitymatching

[8] S. Mudgal et al., “Deep Learning for Entity Matching: A Design Space Exploration,” in Proc. ACM SIGMOD, 2018 (DeepMatcher). https://github.com/anhaidgroup/deepmatcher

[9] Y. Li, J. Li, Y. Suhara, A. Doan and W.-C. Tan, “Deep Entity Matching with Pre-Trained Language Models” (Ditto), PVLDB, 2020. https://arxiv.org/abs/2004.00584

[10] datasketch: MinHash, LSH and other probabilistic data structures for Python. https://github.com/ekzhu/datasketch